NeuralPricer – Deep Learning Based Option Pricing Engine

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF THE DEGREE OF

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

BY

Rajpratap Baniya
Jay Kshirsagar
Dinesh Suthar
John Bright

UNDER THE GUIDANCE OF

Prof. Shubham Bakal

(Department of Information Technology)



INFORMATION TECHNOLOGY DEPARTMENT

XAVIER INSTITUTE OF ENGINEERING

UNIVERSITY OF MUMBAI

2026–2027

Institute Vision

To nurture the joy of excellence in a world of high technology.

Institute Mission

To strive to match global standards in technical education by interaction with industry, continuous staff training and development of quality of life.

Department Vision

To nurture the joy of excellence in the world of Information Technology.

Department Mission

M1: To develop the critical thinking ability of students by promoting interactive learning.

M2: To bridge the gap between industry and institute and give students the kind of exposure to the industrial requirements in current trends of developing technology.

M3: To promote learning and research methods and make them excel in the field of their study by becoming responsible while dealing with social concerns.

M4: To encourage students to pursue higher studies and provide them awareness on various career opportunities that are available.

Program Education Objective (PEO)

PEO1: employed as IT professionals, and shall engage themselves in learning, understanding, and applying newly developed ideas and technologies as their field of study evolves.

PEO2: competent to use the learnt knowledge successfully in the diversified sectors of industry, academia, research and work effectively in a multidisciplinary environment.

PEO3: aware of professional ethics and create a sense of social responsibility in building the nation/society.

Program Specific outcome (PSO)

PSO1: Demonstrate the ability to analyze and visualize the business domain and formulate appropriate information technology solutions.

PSO2: Apply various technologies like Intelligent Systems, Data Mining, IOT, Cloud and Analytics, Computer and Network Security etc. for innovative solutions to real time problems.

Program Outcomes (PO)

PO1: Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.

PO3: Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions for complex problems.

PO5: Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large.

PO11: Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long Learning: Recognize the need for, and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.

XAVIER INSTITUTE OF ENGINEERING

MAHIM CAUSEWAY, MAHIM, MUMBAI - 400016.

CERTIFICATE

This is to certify that

Rajpratap Baniya	06
Jay Kshirsagar	30
Dinesh Suthar	49
John Bright	64

Have satisfactorily carried out the Data Science Lab MINI-PROJECT work titled **"NeuralPricer – Deep Learning Based Option Pricing Engine"** in partial fulfillment of the degree of Bachelor of Engineering as laid down by the University of Mumbai during the academic year 2026–2027.

Internal Examiner/Guide **External Examiner**

Date:

Place: MAHIM, MUMBAI

DECLARATION

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, We have adequately cited and referenced the original sources.

We also declare that We have adhered to all the principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission.

We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which thus have not been properly cited or from whom proper permission have not been taken when needed.

Rajpratap Baniya (06) _____

Jay Kshirsagar (30) _____

Dinesh Suthar (49) _____

John Bright (64) _____

Date:

TABLE OF CONTENTS

SR. NO.	TOPIC	PAGE NO.
A.	ABSTRACT	i
B.	LIST OF FIGURES	ii
C.	LIST OF TABLES	iii
D.	ACKNOWLEDGEMENT	iv
E.	LAB OUTCOMES	v
	RUBRICS	vi
	LO - PO MAPPING	vii
1.	Introduction to Neural Option Pricing 1.1 Option Pricing and Derivatives 1.2 Black–Scholes Model and the Greeks 1.3 Limitations of Classical Pricing Methods 1.4 Crypto Options and the Deribit Exchange 1.5 Working of NeuralPricer 1.6 Real-World Problems and Neural Solutions 1.7 Deep Learning Approach for Option Pricing 1.8 Model Evaluation Metrics 1.9 Applications of NeuralPricer 1.10 Chapter Summary	1
2.	Introduction to NeuralPricer 2.1 Problem Definition 2.2 Aim and Objectives 2.3 Scope of Project 2.4 Features of the Project	7
3.	Review of Literature	9
4.	Implementation Methodology 4.1 Design 4.2 Block Diagram / Flowchart of Proposed System 4.3 Hardware, Software and Final Model Artifacts 4.4 Module Implementation 4.5 Database Connectivity 4.6 Code (All Procedure) 4.7 Output 4.8 Test Cases and Validation 4.9 Conclusion and Future Scope	13
5.	REFERENCES	37
6.	APPENDIX	39

Abstract

NeuralPricer is a deep-learning option pricing engine that replaces repeated evaluation of an analytic pricing formula with a fast neural-network surrogate trained on real exchange data. Live option quotes for Bitcoin (BTC) and Ethereum (ETH) European options were collected from the public Deribit REST API as 79 market snapshots containing 127,322 quotes. After cleaning, 122,471 real quotes were retained and divided *by expiry date*, so that every validation and test contract belongs to a maturity that the network never saw during training. Because the real snapshots cover a very narrow spot-price range, the network is first pre-trained on 278,719 synthetic Black–Scholes samples drawn over a wide range of moneyness, maturity, volatility and rate, and is then fine-tuned on the real quotes.

Black–Scholes pricing is homogeneous in spot and strike, so the model learns the price as a percentage of spot through a scale-free six-feature multilayer perceptron with 110,209 parameters; a single network therefore prices both BTC and ETH. On the 6,754 held-out real quotes the fine-tuned network reached a mean absolute error of USD 80.38 (0.156 % of spot) and $R^2 = 0.9988$, a 31.8 % reduction of the error of the pre-trained network, while Black–Scholes evaluated with the exchange's own mark implied volatility reached USD 23.15. The network is exported as a TorchScript model and served by a Flask REST API with MongoDB-backed authentication and quote history, and a React dashboard provides an option pricer, Greeks, live option chain, price surface and benchmark page. The project is an educational study and does not give investment advice.

LIST OF FIGURES

SR. NO.	FIGURE CAPTION	PAGE NO.
1	Fig. 1.1: Neural Option Pricing Workflow	4
2	Fig. 4.1: Overall System Architecture of NeuralPricer	13
3	Fig. 4.2: Data Preparation and Training Pipeline	14
4	Fig. 4.3: Neural Network Architecture and Export Wrapper	15
5	Fig. 4.4: Google Colab Training Output (Training Curves and NN vs Market Mid Price)	28
6	Fig. 4.5: Test Error of Black–Scholes and the Neural Networks	29
7	Fig. 4.6: Landing Page	31
8	Fig. 4.7: Sign-In Page	31
9	Fig. 4.8: Dashboard – Deribit Index and BTC-Perpetual Chart	32
10	Fig. 4.9: Option Pricer (Neural Network vs Black–Scholes)	32
11	Fig. 4.10: Greeks Page	33
12	Fig. 4.11: Live Option Chain from Deribit	33
13	Fig. 4.12: Expiry-wise Options Chain with NN–Market Gap	34
14	Fig. 4.13: Benchmark Page	34

LIST OF TABLES

SR. NO.	TABLE TITLE	PAGE NO.
1.	Table 3.1: Literature Review Summary	11
2.	Table 3.2: Model Comparison on the Real Test Expiry	12
3.	Table 4.1: Data and Computational Requirements	15
4.	Table 4.2: Final Model Artifacts	16
5.	Table 4.3: Dataset Summary	17
6.	Table 4.4: Data Cleaning Log	17
7.	Table 4.5: Expiry-Based Data Split	18
8.	Table 4.6: Feature Specification	19
9.	Table 4.7: Training Configuration	19
10.	Table 4.8: REST API Endpoints	20
11.	Table 4.9: MongoDB Collections	21
12.	Table 4.10: Model Performance on Real Quotes	29
13.	Table 4.11: Test MAE by Currency and Moneyness	29
14.	Table 4.12: Backend-Path Consistency Check	30
15.	Table 4.13: Representative Test Cases	35

ACKNOWLEDGEMENT

We would like to thank Fr.Dr.John Rose S.J. (Director of XIE) for providing us with such an environment so as to achieve the goals of our project and supporting us constantly.

We express our sincere gratitude to our Honorable Principal Dr. Lata Ragha for encouragement and facilities provided to us.

We would like to place on record our deep sense of gratitude to Dr. Puja Padiya, Head of Department of Information Technology, Xavier Institute of Engineering, Mahim, Mumbai, for generous guidance, help and useful suggestions.

With deep sense of gratitude, we acknowledge the guidance of our project guide **Prof. Shubham Bakal**. The time-to-time assistance and encouragement by her has played an important role in the development of our project.

We would also like to thank our entire Information Technology staff who have willingly cooperated with us in resolving our queries and providing us all the required facilities on time.

RAJPRATAP BANIYA (202303006) _____

JAY KSHIRSAGAR (202303032) _____

DINESH SUTHAR (202303056) _____

JOHN BRIGHT (2024032001) _____

XAVIER INSTITUTE OF ENGINEERING
Department of Information Technology
(NBA Accredited)
(Approved by AICTE, Accredited with "A+" by NAAC, Affiliated to Mumbai University,
Recognized by DTE, Govt. of Maharashtra)
Mahim Causeway, Mahim, Mumbai - 400 016 • Phone : +91 7249371505 /+91 9920407274
Email ID : hod-it@xavier.ac.in • Website : www.xavier.ac.in

CLASS: BE IT **SEM: VII**

COURSE CODE: ITL 701 **COURSE NAME: Data Science Lab**

AY: 2026–2027 **SUB IN-CHARGE: Prof. Shubham Bakal**

LAB OBJECTIVES:

Sr. No.	Lab Objectives
The Lab experiments aims:	
1	To apply reasoning for a problem in an uncertain domain.
2	To discuss the solution after building a Cognitive application.
3	To familiarize the students with the basics of Fuzzy Logic and Fuzzy Systems.
4	To familiarize the students with Learning Architectures and Frameworks
5	To define and apply metrics to measure the performance of various learning algorithms.
6	To enable students to analyze data science methods for real world problems.

Bloom's Taxonomy Levels:

1 = Remembering, 2 = Understanding, 3 = Applying, 4 = Analyzing, 5 = Evaluating, 6 = Creating

LAB OUTCOMES:

Sr. No.	Lab Outcomes	Cognitive Levels
On successful completion, of course, learner/student will be able to:		
1	Implement reasoning with uncertainty.	L1,L2,L3
2	Explore use cases of Cognitive Computing	L1,L2
3	Implement a fuzzy controller system.	L1,L2,L3
4	Develop real life applications using learning concepts.	L1,L2,L3
5	Evaluate performance of applications	L1,L2,L3,L4
6	Implement and analyze applications based on current trends in Data Science.	L1,L2,L3,L4,L5

Course name: Data Science Lab

Group No: 05

Name & Roll No:

1) Rajpratap Baniya - 06
2) Jay Kshirsagar - 30
3) Dinesh Suthar - 49
4) John Bright - 64

Mini Project
LO1: Implement reasoning with uncertainty.
LO2: Explore use cases of Cognitive Computing
LO3: Implement a fuzzy controller system
LO4: Develop real life applications using learning concepts.
LO5: Evaluate performance of applications.
LO6: Implement and analyze applications based on current trends in Data Science

Rubrics For Mini Project Work

Roll No.	**Name of the Student**	**Problem Statement (05)**	**Creativity& Quality of Work done (04)**	**Punctuality & Lab Ethics (02)**	**Performance/ Presentation (04)**	**Total (15)**
06	Rajpratap Baniya					
30	Jay Kshirsagar					
49	Dinesh Suthar					
64	John Bright					

Prof. Shubham Bakal

Subject In-charge

Lab Outcomes:

Sr. No.	Lab Outcomes	Bloom's Taxonomy
On successful completion, of course, learner/student will be able to:		
1	Implement reasoning with uncertainty.	L1,L2,L3
2	Explore use cases of Cognitive Computing	L1,L2
3	Implement a fuzzy controller system.	L1,L2,L3
4	Develop real life applications using learning concepts.	L1,L2,L3
5	Evaluate performance of applications.	L1,L2,L3,L4
6	Implement and analyze applications based on current trends in Data Science.	L1,L2,L3,L4,L5

LO-PO-PSO MAPPING

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
LO1														
LO2														
LO3														
LO4														
LO5														
LO6														

Project topic- PO/PSO Mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
Project Topic														

Chapter 1

INTRODUCTION TO NEURAL OPTION PRICING

1.1 Option Pricing and Derivatives

A financial derivative is a contract whose value depends on the price of an underlying asset. An *option* is the most widely traded derivative: a *call* option gives its holder the right, but not the obligation, to buy the underlying asset at a fixed *strike* price K on or before an *expiry* date, and a *put* option gives the right to sell. A *European* option can be exercised only at expiry. Because the holder can never be forced to exercise, the option has a non-negative payoff

$$\text{Call payoff} = \max(S_T - K, 0), \qquad \text{Put payoff} = \max(K - S_T, 0), \tag{1.1}$$

where S_T is the price of the underlying asset at expiry. The price paid today for this payoff, called the *premium*, depends on the current spot price S, the strike K, the time to maturity T, the risk-free interest rate r and the expected volatility σ of the underlying asset.

Exchanges continuously publish bid and ask quotes for thousands of option contracts, one for every combination of expiry, strike and option type. Traders, market makers and risk managers must therefore evaluate the pricing function for a very large number of contracts every time the market moves. The speed and accuracy of this evaluation is the central practical problem addressed by this project.

1.2 Black–Scholes Model and the Greeks

The Black–Scholes model (Black and Scholes, 1973; Merton, 1973) assumes that the underlying asset follows a geometric Brownian motion with constant volatility. For a

European option on an asset that pays no income it gives the closed-form prices

$$d_1 = \frac{\ln(S/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}, \tag{1.2}$$

$$C = S\,\Phi(d_1) - Ke^{-rT}\Phi(d_2), \tag{1.3}$$

$$P = Ke^{-rT}\Phi(-d_2) - S\,\Phi(-d_1), \tag{1.4}$$

where Φ is the standard normal cumulative distribution function. The sensitivities of the price to its inputs are called the *Greeks*. NeuralPricer reports the five Greeks that are used most often in practice:

- **Delta** $\Delta = \partial V/\partial S$ – sensitivity to the spot price ($\Phi(d_1)$ for a call);
- **Gamma** $\Gamma = \partial^2 V/\partial S^2 = \phi(d_1)/(S\sigma\sqrt{T})$ – rate of change of delta;
- **Vega** $\nu = \partial V/\partial\sigma = S\phi(d_1)\sqrt{T}$ – sensitivity to volatility (reported per 1 % change);
- **Theta** $\Theta = \partial V/\partial t$ – time decay (reported per calendar day);
- **Rho** $\rho = \partial V/\partial r$ – sensitivity to the interest rate (reported per 1 %).

Here ϕ is the standard normal density. Put–call parity, $C - P = S - Ke^{-rT}$, links the call and put prices and is used as one of the unit tests of this project.

1.3 Limitations of Classical Pricing Methods

Although the Black–Scholes formula is fast, its constant-volatility assumption is not observed in real markets. When the formula is inverted to find the volatility implied by a market price, the implied volatility varies with strike and maturity, producing the well-known volatility *smile* and *skew*. More realistic models, such as stochastic-volatility or jump-diffusion models, have no simple closed form and are usually priced by Monte Carlo simulation or numerical PDE solvers. These methods are accurate but need thousands of simulated paths for every contract, which is too slow when thousands of quotes must be refreshed in real time.

Comparison of pricing approaches

Property	**Black–Scholes**	**Monte Carlo**	**Neural surrogate**
Evaluation speed	Very fast	Slow (many paths)	Fast (one forward pass, batchable)
Needs closed form	Yes	No	No
Learns market behaviour	No	Only if model is rich	Yes, from market data
Training cost	None	None	One-off, offline
Differentiable	Analytic Greeks	Bumping / pathwise	Automatic differentiation

1.4 Crypto Options and the Deribit Exchange

Deribit is a derivatives exchange on which European-style options on Bitcoin (BTC) and Ethereum (ETH) are traded. Its public REST API provides, without authentication, the index (spot) price, the list of active option instruments and a book summary containing the bid price, ask price and mark implied volatility of every contract. Instrument names follow the pattern `BTC-27NOV26-73000-P`, which encodes the currency, expiry date, strike and option type. Option prices are quoted in units of the underlying coin and are converted to US dollars by multiplying by the index price. Crypto options are attractive for a data-science project because they are open for trading continuously, publish a large number of contracts at many maturities, and the data are freely available.

1.5 Working of NeuralPricer

NeuralPricer treats option pricing as a supervised regression problem. A neural network receives the contract features $(S, K, T, r, \sigma, \text{type})$ and predicts the premium. The network is trained offline; at run time a single forward pass returns the price. The workflow is shown in Fig. 1.1.

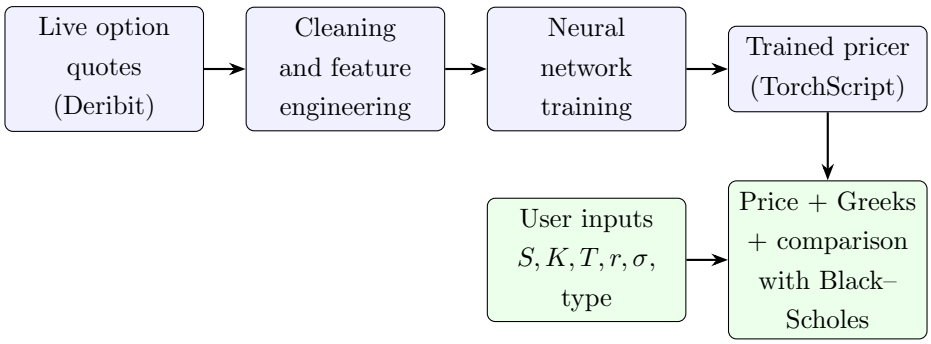


Figure 1.1: Neural Option Pricing Workflow

1.6 Real-World Problems and Neural Solutions

Building a neural option pricer from market data raises several practical problems, each of which is addressed in the project.

- **Narrow market coverage.** The 79 collected snapshots span a spot-price range of only about 1.3 % for BTC and 1.0 % for ETH. A network trained on such data cannot price an option when the spot has moved. The project therefore pre-trains on synthetic Black–Scholes data generated over a wide range ($0.7S_0$ to $1.3S_0$), and fine-tunes on the real quotes.
- **Scale differences between assets.** BTC trades near USD 82 000 and ETH near USD 2 500. Because the Black–Scholes price is homogeneous in (S, K), the network learns the price as a fraction of spot, which lets one model serve both assets.
- **Data leakage.** The same contracts reappear in every snapshot, so a random row-wise split would place almost identical rows in training and test sets and give over-optimistic results. The project holds out *whole expiries*.
- **Noisy quotes.** One-sided quotes and extremely wide bid–ask spreads are removed before training.
- **Heavy-tailed prices.** Prices range from under USD 1 to above USD 20 000. A log-transformed target is used so that cheap out-of-the-money options are not ignored by the loss function.
- **Deployment mismatch.** The exported model accepts exactly the nine scaled features built by the backend, and a consistency check confirms that the backend path reproduces the notebook predictions.

1.7 Deep Learning Approach for Option Pricing

The pricing function is approximated by a feed-forward multilayer perceptron (MLP) with batch normalisation and ELU activations. Training happens in two stages. In *Stage 1* the network is pre-trained on synthetic samples whose labels are exact Black–Scholes prices, so it learns the correct shape of the pricing surface over a wide range of inputs. In *Stage 2* the network is fine-tuned on real Deribit quotes (mixed with an equal number of replayed synthetic samples to prevent forgetting) with a smaller learning rate, so that it adapts to the real market while keeping its global behaviour. Early stopping and model selection use validation data from *unseen expiries*.

1.8 Model Evaluation Metrics

Because the target is a continuous price, regression metrics are used. For true prices y_i and predictions $\hat{y}_i$ over n test quotes:

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}, \tag{1.5}$$

$$\mathrm{MAE}_{\%S} = \frac{1}{n}\sum_{i=1}^{n}\frac{|y_i - \hat{y}_i|}{S_i} \times 100, \qquad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \tag{1.6}$$

MAE and RMSE are reported in US dollars, $\mathrm{MAE}_{\%S}$ expresses the error as a percentage of the spot price so that BTC and ETH can be compared, and the median relative error (computed for options worth at least 0.5 % of spot) shows the typical percentage error on meaningful prices. Lower error values and higher R^2 are better.

1.9 Applications of NeuralPricer

- **Real-time pricing of large option chains:** a batch of hundreds of contracts is priced in one forward pass.
- **Market screening:** the gap between the network price and the exchange mid price highlights contracts that look rich or cheap relative to the learned surface (a model signal, not a trading recommendation).
- **Risk analysis:** Greeks and price surfaces over strike and maturity help to understand exposure.

- **Education:** the dashboard compares the neural pricer with Black–Scholes side by side, which makes the model easy to explain.
- **Template for other assets:** the same pipeline can be applied to equity options (yfinance) or other derivatives that lack a closed-form price.

1.10 Chapter Summary

This chapter introduced option pricing, the Black–Scholes model and the Greeks, the limitations of classical methods, and the idea of a neural-network surrogate trained on real Deribit data. The next chapter defines the problem, aim, scope and features of NeuralPricer.

Chapter 2

INTRODUCTION TO NEURALPRICER

2.1 Problem Definition

The project is a supervised regression system for European options on BTC and ETH. Given the spot price S, strike K, time to maturity T, risk-free rate r, volatility σ and option type, it must predict the option premium in US dollars quickly and accurately, including for market states that were not seen in the limited real data. The supplied real data consist of 79 snapshots taken within a short window, in which the same contracts repeat and the spot price barely moves. A usable system therefore has to (i) avoid leakage in evaluation, (ii) generalise beyond the narrow real range, and (iii) be integrated into a working application.

2.2 Aim and Objectives

The aim of this project is to build and evaluate a deep-learning option pricing engine, trained on real exchange data, and to deploy it as a web application.

The objectives of the project are:

- To collect live BTC and ETH option snapshots from the Deribit public API and store them as Parquet files.
- To audit, clean and engineer features from the snapshots without introducing leakage.
- To generate a wide-range synthetic Black–Scholes dataset for pre-training.
- To design a scale-free neural network so that one model prices both BTC and ETH.
- To fine-tune on real quotes and evaluate on unseen expiries against Black–Scholes using mark implied volatility.

- To export the model as TorchScript and serve it through a Flask REST API with MongoDB authentication and quote logging.
- To build a React dashboard with an option pricer, Greeks, live chain, price surface and benchmark pages.

2.3 Scope of Project

The scope covers European-style BTC and ETH options on Deribit with a fixed risk-free rate of 4.5 % and expiries of at least one day. Real data were collected in a single short period, so the model has seen the real market at one regime only; the synthetic stage is used to cover other spot levels, and results must not be read as live trading performance. Greeks in the application are computed analytically from Black–Scholes. The project is educational: model prices are estimates, not investment advice or a forecast of market direction. American options, dividends, and equity options are outside the present scope.

2.4 Features of the Project

- Live collection of BTC and ETH option snapshots from Deribit (bulk book-summary request).
- Expiry-based train/validation/test split that prevents leakage between snapshots.
- Two-stage learning: synthetic Black–Scholes pre-training and real-market fine-tuning.
- Scale-free 110,209-parameter MLP shared by BTC and ETH.
- TorchScript export with an embedded pre- and post-processing wrapper, compatible with the backend feature pipeline.
- REST API: single price, batch price, Greeks, live option chain, authentication and history.
- MongoDB-backed user accounts, 30-day session tokens and price-quote logging.
- React dashboard: landing page, sign-in, Deribit dashboard, option pricer, Greeks, live chain, price surface and benchmark pages.
- Side-by-side neural vs Black–Scholes comparison and a live API latency benchmark.

Chapter 3

REVIEW OF LITERATURE

Introduction to Literature Review

Neural networks have been used to price derivatives for three decades. This chapter first summarises the published work that motivates NeuralPricer and then reviews the three pricing approaches that were compared experimentally in this project, in the same manner in which model experiments are reviewed in the reference report.

Paper 1: A Nonparametric Approach to Pricing and Hedging Derivative Securities via Learning Networks

Hutchinson, Lo and Poggio (1994) trained learning networks to learn the Black–Scholes pricing function from simulated data and showed that the networks could also price and hedge on real S&P 500 futures options. This is the earliest well-known demonstration that a network can act as a nonparametric pricing function; NeuralPricer follows the same idea with modern deep-learning practice.

Paper 2: Machine Learning in Finance: The Case of Deep Learning for Option Pricing

Culkin and Das (2017) trained a deep feed-forward network on simulated Black–Scholes prices over a grid of inputs and reported that the network reproduced option prices with very small errors. This supports the use of synthetic Black–Scholes data as a pre-training source, as done in Stage 1 of this project.

Paper 3: Pricing Options and Computing Implied Volatilities Using Neural Networks

Liu, Oosterlee and Bohte (2019) used neural networks both for the forward problem (price from parameters) and for the inverse problem (implied volatility from price), and demonstrated that the expensive numerical step can be moved offline into the training stage so that evaluation becomes fast.

Paper 4: Deep Learning Volatility

Horvath, Muguruza and Tomas (2021) proposed learning the mapping from model parameters to the implied-volatility surface offline with a neural network, which makes calibration of rich models fast. This motivates the future-scope direction of learning the volatility surface directly.

Paper 5: Neural Networks for Option Pricing and Hedging: A Literature Review

Ruf and Wang (2020) reviewed the use of neural networks for option pricing and hedging and discuss design choices such as input features, outputs, loss functions and the importance of how data are split and evaluated, which motivates the expiry-based evaluation and scale-free features used here.

Paper 6: Physics-Informed Neural Networks

Raissi, Perdikaris and Karniadakis (2019) introduced physics-informed neural networks that add a differential-equation residual to the training loss. Applied to option pricing, the Black–Scholes PDE can act as a regulariser. The idea is listed as future work for NeuralPricer.

Table 3.1: Literature Review Summary

No.	Reference	Idea	Relevance to NeuralPricer
1	Hutchinson, Lo and Poggio (1994)	Learning networks as nonparametric pricing functions	Foundation of neural surrogate pricing
2	Culkin and Das (2017)	Deep network trained on Black–Scholes samples	Justifies synthetic pre-training
3	Liu, Oosterlee and Bohte (2019)	Neural networks for price and implied volatility	Offline training, fast online evaluation
4	Horvath, Muguruza and Tomas (2021)	Offline learning of the volatility surface	Future scope: surface model
5	Ruf and Wang (2020)	Review of neural pricing and hedging	Feature, loss and evaluation design
6	Raissi, Perdikaris and Karniadakis (2019)	PDE-constrained training	Future scope: PINN loss

Model 1: Black–Scholes with Mark Implied Volatility (Benchmark)

The first approach evaluates the analytic Black–Scholes formula using the mark implied volatility published by Deribit for each contract. Since the exchange derives this volatility from its own pricing of the same contract, this benchmark is extremely strong: it receives the market's own volatility as an input, which the neural network is not asked to predict. On the 6,754 held-out real quotes it achieved MAE USD 23.15, RMSE USD 39.52 and $R^2 = 0.9999$.

Model 2: Neural Network Pre-trained Only

The MLP is trained only on 250,848 synthetic Black–Scholes samples (Stage 1). It learns the pricing surface over a wide range, but has no exposure to real-market effects. On the real test expiry it reached MAE USD 117.82, RMSE USD 219.34 and $R^2 = 0.9974$, with a median relative error of 5.66 %.

Model 3: Neural Network Fine-Tuned on Real Quotes

The pre-trained network is fine-tuned on the 95,090 real training quotes mixed with an equal number of replayed synthetic samples (Stage 2). On the real test expiry it reached MAE USD 80.38, RMSE USD 147.67 and $R^2 = 0.9988$, with a median relative error of 3.39 %. On the validation expiries the fine-tuned network also had a lower MAE (USD 122.29 against USD 129.27) and was therefore selected for export.

Table 3.2: Model Comparison on the Real Test Expiry (6,754 quotes)

Model	**MAE (USD)**	**RMSE (USD)**	**MAE (% spot)**	**Med. err (%)**	R^2
BS (mark IV)	23.15	39.52	0.0415	0.92	0.9999
NN pre-trained only	117.82	219.34	0.2758	5.66	0.9974
NN fine-tuned	80.38	147.67	0.1561	3.39	0.9988

Conclusion of Literature Review

The literature shows that neural networks can approximate pricing functions and that synthetic pre-training and careful evaluation are important. The experiments of this project show that fine-tuning on real quotes clearly improves on the purely synthetic network (MAE reduced by 31.8 % on the test expiry), whereas Black–Scholes with the exchange's mark volatility remains the most accurate on real quotes because it is given the volatility as an input. The contribution of the neural pricer is therefore a fast, differentiable, single-model surrogate that needs only $(S, K, T, r, \sigma, \text{type})$, not a replacement for the analytic formula where the formula applies.

Chapter 4

IMPLEMENTATION METHODOLOGY

4.1 Design

NeuralPricer is organised in four layers. The *data layer* collects Deribit option snapshots and stores them as Parquet files on Google Drive. The *learning layer* (a Google Colab notebook on a T4 GPU) audits and cleans the data, generates synthetic Black–Scholes data, trains the network in two stages and exports TorchScript and scaler artifacts. The *service layer* is a Flask REST API that loads the exported model, prices requests, computes Greeks, fetches the live option chain and stores users, sessions and quote history in MongoDB. The *presentation layer* is a React single-page dashboard that calls the API. The complete architecture is shown in Fig. 4.1.

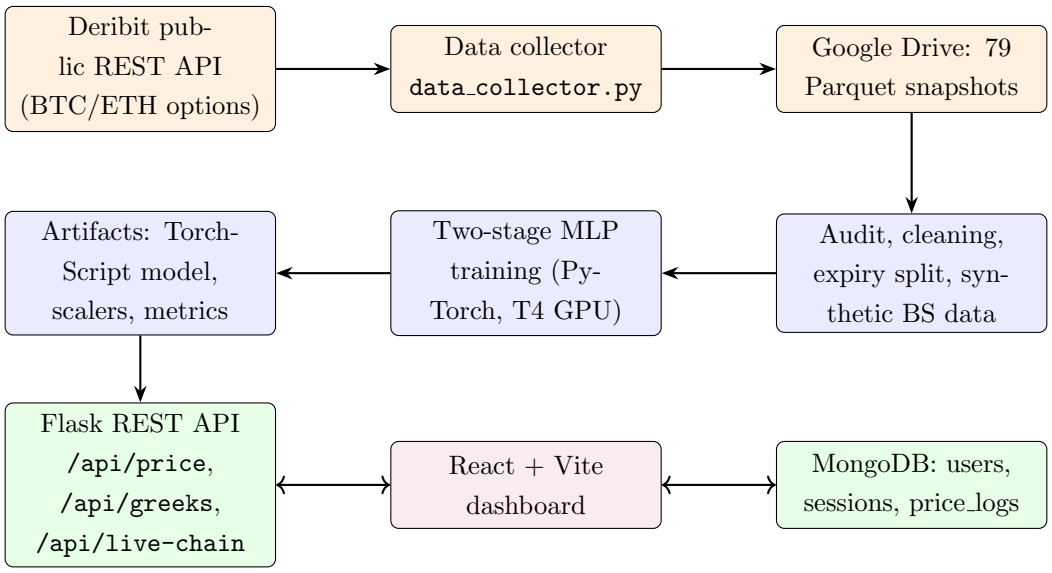


Figure 4.1: Overall System Architecture of NeuralPricer

4.2 Block Diagram / Flowchart of Proposed System

The data preparation and training procedure is shown in Fig. 4.2. Each block corresponds to a cell of the Colab notebook `NeuralPricer_Train_Colab.ipynb`.

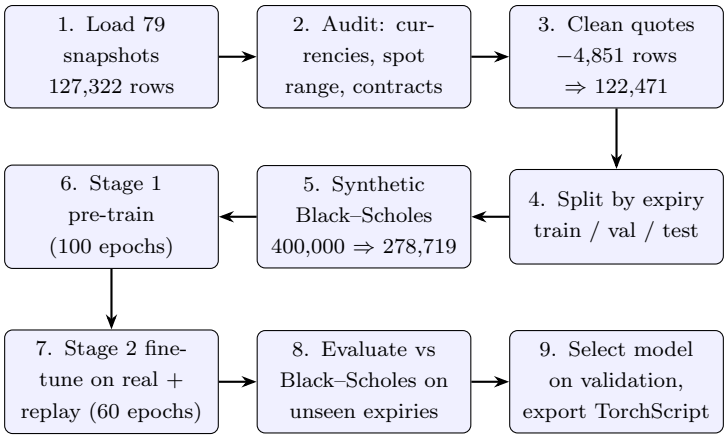


Figure 4.2: Data Preparation and Training Pipeline

4.2.1 Scale-free formulation

For European options the Black–Scholes price is homogeneous of degree one in the pair (S, K):

$$V(S, K, T, r, \sigma) = S \cdot f\left(\ln \frac{S}{K},\, T,\, r,\, \sigma\right). \tag{4.1}$$

The network therefore learns the dimensionless function f and the dollar price is recovered by multiplying with S. This is what allows a single network to price both BTC (spot $\approx 82\,000$) and ETH (spot $\approx 2\,500$), and to price at spot levels outside the narrow real range. Because option prices span several orders of magnitude, the regression target is the logarithm of the price expressed in percent of spot:

$$y = \ln\left(100\,\frac{V}{S} + \varepsilon\right), \qquad \varepsilon = 0.01, \qquad \hat{V} = S \cdot \frac{\max\left(e^{\hat{y}} - \varepsilon,\, 0\right)}{100}. \tag{4.2}$$

The six inputs of the core network are $\ln(S/K)$, $\sqrt{T}$, σ, r, the call/put flag and $\sigma\sqrt{T}$ (total volatility), standardised with the mean and standard deviation of the synthetic training set.

4.2.2 Network architecture

The core network (Fig. 4.3) has four hidden blocks of sizes 256, 256, 128 and 64, each consisting of a linear layer, batch normalisation and an ELU activation, followed by a

linear output unit. Dropout is not used because the task is a deterministic regression surrogate. The network has 110,209 trainable parameters. A `Pricer` wrapper module stores the scaler statistics as buffers, accepts the *same nine scaled features* that the backend builds, reconstructs the six core features internally and returns the price in US dollars; the wrapper is traced with `torch.jit.trace` so that the backend and the scalers files need no change.

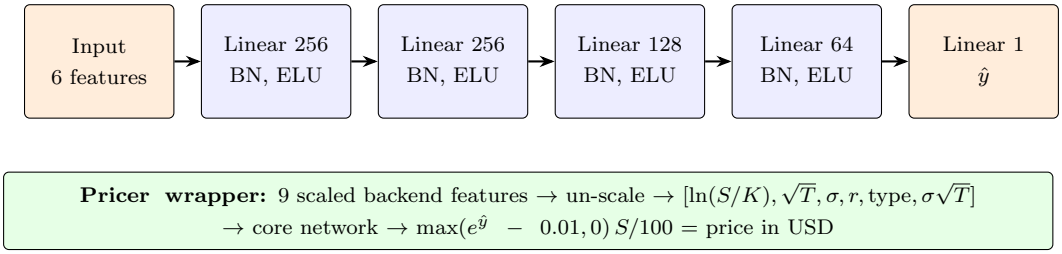


Figure 4.3: Neural Network Architecture and Export Wrapper

4.3 Hardware, Software and Final Model Artifacts

Table 4.1: Data and Computational Requirements

Sr. No.	Requirement
1	Deribit public REST API access (no authentication) for BTC and ETH option book summaries and index prices
2	Google Colab with a T4 GPU for training (about 5–10 minutes per run); Google Drive for snapshot and model storage
3	Python 3.10+ with PyTorch 2.x, NumPy, pandas, PyArrow, SciPy, scikit-learn and joblib
4	Flask 3.x, Flask-CORS and PyMongo for the REST API; MongoDB Community Server 7.x
5	Node.js with React 18, Vite, Tailwind CSS, Recharts, axios and lucide-react for the dashboard
6	A computer able to run Python, Node.js and MongoDB; no GPU is required to run the trained model (CPU inference)
7	Git and GitHub for version control

Table 4.2: Final Model Artifacts

Sr. No.	Artifact	Purpose
1	`best_model_scripted.pt` (495,523 bytes)	TorchScript `Pricer` loaded by the backend
2	`best_model.pt` (456,245 bytes)	State dictionary of the core network
3	`scaler_X.pkl`	Standard scaler for the nine backend features
4	`scaler_y.pkl` (534 bytes)	Identity scaler (the model already returns USD)
5	`metrics.json`	Configuration, cleaning log, split sizes and all metrics
6	`history.json`	Training and validation loss of both stages
7	`eval.png`	Training curves and NN-vs-market scatter plot

4.4 Module Implementation

4.4.1 Data collection module

`data_collector.py` calls `get_index_price` for the spot price and `get_book_summary_by_currency` once per currency to obtain every active option in a single request. For each contract it parses the instrument name into strike, option type and expiry (Deribit options settle at 08:00 UTC), computes the time to maturity in years, converts the coin-denominated bid, ask and mid price to US dollars, takes the mark implied volatility (given in percent) as σ, and fixes $r = 4.5\,\%$. A loop saves each snapshot as `snap_<timestamp>.parquet`; this was run to collect 79 snapshots on Google Drive.

4.4.2 Audit and cleaning module

The notebook loads the snapshots, adds a snapshot number, derives currency and expiry from the instrument name and prints an audit (Table 4.3). The same 1,748 contracts repeat in every snapshot, and the spot price varies by only about 1 %; these two facts drive the design of the split and of the synthetic stage. Quotes are cleaned according to Table 4.4.

Table 4.3: Dataset Summary

Item	**Value**
Snapshots	79 (1,602 to 1,637 quotes each, mean 1,612)
Raw rows	127,322 (BTC 68,279; ETH 59,043)
Unique contracts	1,748 (repeated in every snapshot)
BTC spot range	USD 82,334 – 83,388 (1.28 % range)
ETH spot range	USD 2,488 – 2,512 (0.97 % range)
Time to maturity	0.0027 – 0.959 years (12 expiries per currency)
Mark implied volatility	BTC 0.24 – 1.12; ETH 0.28 – 1.25
Rows after cleaning	122,471
Synthetic samples	400,000 generated; 278,719 kept (250,848 train, 27,871 validation)

Table 4.4: Data Cleaning Log

Rule	**Rows removed**
Missing values in core fields	0
Implied volatility outside 5 % – 300 %	0
Time to maturity below one day	0
One-sided quote (bid or ask equal to zero)	4,584
Bid–ask spread wider than 100 % of mid	267
Mid price below tick-noise floor (0.015 % of spot)	0
Mid price not below spot	0
Total removed	**4,851**

4.4.3 Expiry-based splitting module

For each currency the expiries are sorted by maturity and the 4th and 11th are assigned to validation and the 6th to test (index modulo 7); the remaining nine expiries are used for training. Whole expiries are held out, so no validation or test contract is ever seen in

training (Table 4.5). Validation expiries have maturities between 0.010 and 0.710 years; the test expiry has $T \approx 0.038$ years (about two weeks).

Table 4.5: Expiry-Based Data Split

Split	**Currency**	**Rows**	**Expiries**	T **range (years)**
Train	BTC	50,822	9	0.003 – 0.959
Train	ETH	44,268	9	0.003 – 0.959
Validation	BTC	11,236	2	0.010 – 0.710
Validation	ETH	9,391	2	0.010 – 0.710
Test	BTC	3,755	1	0.038 – 0.039
Test	ETH	2,999	1	0.038 – 0.039
Total		**122,471**		

4.4.4 Synthetic data module

Synthetic samples are drawn from ranges derived from the real training data: $\ln(K/S) \in [-1.54, 1.54]$, $T \in [0.0027, 1.199]$ years (log-uniform), $\sigma \in [0.20, 1.19]$, $r \in [0.02, 0.06]$, and $S = S_0 \cdot U(0.7, 1.3)$ with S_0 the median real spot of BTC (82,535) or ETH (2,499), selected with the same BTC/ETH share as the training data. Exact Black–Scholes prices are the labels. Samples whose price is below 0.015 % of spot are dropped, leaving 278,719 of 400,000.

4.4.5 Feature engineering module

Table 4.6: Feature Specification

Feature	**Definition**	**Used by**
`S`, `K`, `T`, `r`, `sigma`, `is_call`	Spot, strike, years to expiry, rate, implied volatility, 1 for call / 0 for put	Backend input (9 scaled features)
`moneyness`	S/K	Backend input
`log_moneyness`	$\ln(S/K)$	Backend input; core feature 1
`sqrt_T`	$\sqrt{T}$	Backend input; core feature 2
$\sigma\sqrt{T}$	Total volatility over life of option	Core feature 6 (built inside the wrapper)

4.4.6 Model training module

Table 4.7: Training Configuration

Setting	**Value**
Network	MLP 6–256–256–128–64–1, BatchNorm, ELU, no dropout
Loss	Huber loss, $\delta = 0.5$, on the log-percent target
Optimiser	AdamW, weight decay 10^{-4}
Learning-rate schedule	Cosine annealing to 1 % of the initial rate
Batch size	4,096
Stage 1 (synthetic)	100 epochs, learning rate 2×10^{-3}
Stage 2 (real + replay)	60 epochs, learning rate 2×10^{-4}, replay ratio 1.0
Early stopping	Patience 30 epochs on validation loss; best weights restored
Best validation loss	Stage 1: 0.00137 (synthetic); Stage 2: 0.00114 (real, unseen expiries)
Random seed	42

In each fine-tuning epoch the real training quotes are concatenated with an equal number of randomly replayed synthetic samples. After training, the model with the lower validation MAE (as percentage of spot) is exported; the fine-tuned network was selected (0.2755 % against 0.2834 %).

4.4.7 Backend service module

The Flask application (`backend/main.py`) loads the TorchScript model and scalers lazily, computes the nine features of each request, runs one forward pass, and computes the analytic Black–Scholes price and Greeks for comparison. If PyTorch or the model files are missing, it falls back to Black–Scholes only. The endpoints are listed in Table 4.8.

Table 4.8: REST API Endpoints

Method	**Endpoint**	**Function**
GET	`/api/health`	Service, model and MongoDB status
POST	`/api/price`	NN price, BS price, absolute and percentage error, Greeks; logs the quote
POST	`/api/price/batch`	Prices a list of contracts (used by the surface page)
POST	`/api/greeks`	Analytic Delta, Gamma, Vega, Theta, Rho
GET	`/api/live-chain/<currency>`	Live Deribit chain for BTC or ETH with the NN price of each contract
POST	`/api/auth/register`, `/api/auth/login`	Create account / sign in, returns session token
POST, GET	`/api/auth/logout`, `/api/auth/me`	End session / current user
GET	`/api/history`	Quote history of the signed-in user

4.4.8 Frontend module

The React dashboard (Vite, Tailwind CSS, Recharts) contains a landing page, sign-up and sign-in pages, and six authenticated tabs: *Dashboard* (Deribit BTC and ETH index and candlestick chart), *Option Pricer* (NN versus Black–Scholes with live sliders), *Greeks* (five Greeks versus spot), *Live Chain* (Deribit chain with NN price and NN-vs-market gap),

Price Surface (strike × maturity heat-map through `/api/price/batch`) and *Benchmark* (stored test metrics and a 30-call latency test).

4.5 Database Connectivity

User management and history use MongoDB 7.x through PyMongo. The connection string and database name come from the environment variables `MONGO_URI` (default `mongodb://127.0.0.1:27017/`) and `MONGO_DB` (default `neuralpricer`). Passwords are stored as Werkzeug password hashes, and a login creates a random 64-character session token that expires after 30 days. If MongoDB is not running, the API still serves prices; only authentication and history are disabled.

Table 4.9: MongoDB Collections

Collection	**Content**	**Index**
`users`	Name, e-mail, hashed password	Unique on `email`
`sessions`	Token, e-mail, name, created, expires	Unique on `token`; TTL on `expires`
`price_logs`	Timestamp, user, S, K, T, r, σ, type, BS price, NN price	Descending on `ts`

4.6 Code (All Procedure)

The principal code of each module is listed below. The complete source (Colab notebook, backend, frontend, training script and tests) is in the project repository and in the submitted `code` folder.

4.6.1 Black–Scholes price and Greeks (`backend/black_scholes.py`)

Listing 4.1: Analytic Black–Scholes price and Greeks

```
import numpy as np
from scipy.stats import norm


def bs_price_and_greeks(S, K, T, r, sigma, is_call):
    """
    Compute Black-Scholes price and all five Greeks.
    S       : spot price
    K       : strike price
```

```
    T       : time to maturity in years
    r       : risk-free rate (decimal, e.g. 0.045)
    sigma   : implied volatility (decimal, e.g. 0.72)
    is_call : 1 for call, 0 for put
    Returns : dict with keys 'price' and 'greeks'
    """
    T     = max(float(T),     1e-8)
    sigma = max(float(sigma), 1e-8)
    S, K, r = float(S), float(K), float(r)

    d1 = (np.log(S / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)

    if is_call == 1:
        price = S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
        delta = float(norm.cdf(d1))
        rho   = float(K * T * np.exp(-r * T) * norm.cdf(d2)  / 100)
        theta = float((-S * norm.pdf(d1) * sigma / (2 * np.sqrt(T))
                       - r * K * np.exp(-r * T) * norm.cdf(d2)) / 365)
    else:
        price = K * np.exp(-r * T) * norm.cdf(-d2) - S * norm.cdf(-d1)
        delta = float(norm.cdf(d1) - 1)
        rho   = float(-K * T * np.exp(-r * T) * norm.cdf(-d2) / 100)
        theta = float((-S * norm.pdf(d1) * sigma / (2 * np.sqrt(T))
                       + r * K * np.exp(-r * T) * norm.cdf(-d2)) / 365)

    gamma = float(norm.pdf(d1) / (S * sigma * np.sqrt(T)))
    vega  = float(S * norm.pdf(d1) * np.sqrt(T) / 100)

    return {
        "price": float(price),
        "greeks": {
            "delta": delta,
            "gamma": gamma,
            "vega":  vega,
            "theta": theta,
            "rho":   rho,
        }
    }
```

4.6.2 Deribit snapshot rows (backend/data_collector.py)

Listing 4.2: Conversion of Deribit book summary into dataset rows

```
def _rows_from_summary(summary, spot, risk_free_rate, currency, now_utc):
    """Convert bulk book-summary records into dataset rows (guide schema)."""
    rows = []
    for item in summary:
        name = item.get("instrument_name")
        bid = item.get("bid_price")
        ask = item.get("ask_price")
        mark_iv = item.get("mark_iv")
        if not name or bid is None or ask is None or not mark_iv:
            continue

        parsed = _parse_instrument_name(name)
        if parsed is None:
            continue
        strike, is_call, expiry_dt = parsed
```

```
        T = (expiry_dt - now_utc).total_seconds() / (365 * 24 * 3600)
        if T <= 0:
            continue

        mid_price = (bid + ask) / 2 * spot
        rows.append({
            "instrument":    name,
            "S":             spot,
            "K":             strike,
            "T":             T,
            "r":             risk_free_rate,
            "sigma":         mark_iv / 100.0,   # Deribit gives IV in %
            "is_call":       is_call,
            "mid_price":     mid_price,
            "bid_usd":       bid * spot,
            "ask_usd":       ask * spot,
            "volume":        item.get("volume", 0),
            "open_interest": item.get("open_interest", 0),
            "snapshot_time": now_utc.isoformat(),
            "currency":      currency,
        })
    return rows
```

4.6.3 Cleaning and expiry-based split (Colab cell 3)

Listing 4.3: Cleaning rules and expiry-based split

```
# Cell 3 - Clean, add features, split by EXPIRY (not by row)
FEATURE_COLS = ["S", "K", "T", "r", "sigma", "is_call", "moneyness", "log_moneyness", "sqrt_T"]

def clean_real(d):
    d, log = d.copy(), []
    for c in NEED: d[c] = pd.to_numeric(d[c], errors="coerce")
    def step(name, keep):
        nonlocal d
        n = len(d); d = d[keep(d)]; log.append((name, n - len(d)))
    step("missing values in core fields", lambda x: x[NEED].notna().all(axis=1))
    step("sigma outside 5%..300%", lambda x: x.sigma.between(0.05, 3.0))
    step("T < 1 day", lambda x: x["T"] >= 1 / 365)
    if {"bid_usd", "ask_usd"} <= set(d.columns):
        step("one-sided quote (bid or ask = 0)", lambda x: (x.bid_usd > 0) & (x.ask_usd > 0))
        step("bid-ask spread too wide", lambda x: (x.ask_usd - x.bid_usd) / x.mid_price <= CFG["max_spread"])
    step("mid price below tick-noise floor", lambda x: x.mid_price / x.S >= CFG["min_frac"])
    step("mid price >= spot", lambda x: x.mid_price < x.S)
    d["moneyness"] = d.S / d.K
    d["log_moneyness"] = np.log(d.moneyness)
    d["sqrt_T"] = np.sqrt(d["T"])
    return d[np.isfinite(d.log_moneyness)].reset_index(drop=True), log

real, clean_log = clean_real(raw)
print(f"Rows kept: {len(real):,} of {len(raw):,}")
for name, n in clean_log: print(f"  removed {n:>7,}  {name}")

def assign_split(d):
    sp = pd.Series("train", index=d.index)
    for cur, g in d.groupby("currency"):
        exps = sorted(g.expiry.unique(), key=lambda e: g.loc[g.expiry == e, "T"].mean())  # by maturity
        mod, v, t = (7, 3, 5) if len(exps) >= 7 else (3, 1, 2)    # spread val/test across maturities
        for i, e in enumerate(exps):
```

```
            if i % mod in (v, t):
                sp[(d.currency == cur) & (d.expiry == e)] = "val" if i % mod == v else "test"
    return sp

real["split"] = assign_split(real)
print("\nSplit (whole expiries are held out, so val/test contracts are never seen in training):")
print(real.groupby(["split", "currency"]).agg(rows=("S", "size"), expiries=("expiry", "nunique"),
      T_min=("T", "min"), T_max=("T", "max")).round(3))
real_tr, real_va, real_te = [real[real.split == s].reset_index(drop=True) for s in ("train", "val", "test")]
assert min(len(real_tr), len(real_va), len(real_te)) > 0, "A split is empty - too few expiries in the data"
real.to_parquet(f"{BASE}/data/processed/real_clean.parquet", index=False)
```

4.6.4 Synthetic Black–Scholes data (Colab cell 4)

Listing 4.4: Wide-range synthetic data generation

```
# Cell 4 - Synthetic Black-Scholes data on a WIDE range (fixes the narrow-spot problem)
def bs_price(S, K, T, r, sigma, is_call):
    T = np.clip(T, 1e-6, None); sigma = np.clip(sigma, 1e-6, None)
    d1 = (np.log(S / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * np.sqrt(T)); d2 = d1 - sigma * np.sqrt(T)
    call = S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
    put = K * np.exp(-r * T) * norm.cdf(-d2) - S * norm.cdf(-d1)
    return np.where(is_call == 1, call, put)

def synth_ranges(tr, allreal):
    q = lambda s, p: float(s.quantile(p))
    x = np.log(tr.K / tr.S); xl, xh = q(x, .005), q(x, .995); pad = 0.25 * (xh - xl)
    return dict(x=(xl - pad, xh + pad),
                T=(1 / 365, q(tr["T"], .995) * 1.25),
                sigma=(max(0.05, q(tr.sigma, .005) * 0.8), q(tr.sigma, .995) * 1.25),
                r=(0.02, 0.06),
                spot0=allreal.groupby("currency").S.median().to_dict(),
                share=tr.currency.value_counts(normalize=True).to_dict())

def make_synth(n, R, seed):
    rng = np.random.default_rng(seed)
    curs = list(R["share"]); cur = rng.choice(curs, n, p=[R["share"][c] for c in curs])
    S = np.array([R["spot0"][c] for c in cur]) * rng.uniform(CFG["spot_lo"], CFG["spot_hi"], n)
    K = S * np.exp(rng.uniform(*R["x"], n))
    T = np.exp(rng.uniform(np.log(R["T"][0]), np.log(R["T"][1]), n))
    sg, r, ic = rng.uniform(*R["sigma"], n), rng.uniform(*R["r"], n), rng.integers(0, 2, n)
    d = pd.DataFrame(dict(currency=cur, S=S, K=K, T=T, r=r, sigma=sg, is_call=ic,
                          mid_price=bs_price(S, K, T, r, sg, ic)))
    d = d[d.mid_price / d.S >= CFG["min_frac"]].copy()
    d["moneyness"] = d.S / d.K; d["log_moneyness"] = np.log(d.moneyness); d["sqrt_T"] = np.sqrt(d["T"])
    return d.reset_index(drop=True)

R = synth_ranges(real_tr, real)
print({k: (tuple(round(float(a), 4) for a in v) if isinstance(v, tuple) else v) for k, v in R.items()})
synth = make_synth(CFG["n_synth"], R, CFG["seed"])
n_va = len(synth) // 10
synth_va, synth_tr = synth.iloc[:n_va].reset_index(drop=True), synth.iloc[n_va:].reset_index(drop=True)
print(f"Synthetic: train {len(synth_tr):,} | val {len(synth_va):,}")
```

4.6.5 Model and export wrapper (Colab cell 5)

Listing 4.5: Scale-free core network and Pricer wrapper

```
# Cell 5 - Model: scale-free core + wrapper that keeps backend/main.py unchanged
# Black-Scholes is homogeneous: price(S,K,...) = S * f(ln(S/K), T, sigma, r, type).
# The core learns f (target = log of price as % of spot), so BTC and ETH share one model.
def core_feats(d):
    S, K, T, sg = d.S.values, d.K.values, d["T"].values, d.sigma.values
    return np.column_stack([np.log(S / K), np.sqrt(T), sg, d.r.values,
                            d.is_call.values.astype(float), sg * np.sqrt(T)]).astype(np.float32)

def target(d): return np.log(100 * d.mid_price.values / d.S.values + EPS).astype(np.float32)

class Core(nn.Module):
    def __init__(self, d_in=6, hidden=(256, 256, 128, 64), p=0.0):   # no dropout: this is a regression surrogate
        super().__init__()
        layers, prev = [], d_in
        for h in hidden:
            layers += [nn.Linear(prev, h), nn.BatchNorm1d(h), nn.ELU(), nn.Dropout(p)]; prev = h
        layers.append(nn.Linear(prev, 1)); self.net = nn.Sequential(*layers)
    def forward(self, x): return self.net(x).squeeze(-1)

class Pricer(nn.Module):
    """Takes the SAME scaled 9 features backend/main.py builds, returns the USD price (N,1)."""
    def __init__(self, core, c_mean, c_std, sx_mean, sx_scale):
        super().__init__(); self.core = core
        for n, v in dict(c_mean=c_mean, c_std=c_std, sx_mean=sx_mean, sx_scale=sx_scale).items():
            self.register_buffer(n, torch.tensor(v, dtype=torch.float32))
    def forward(self, xs):
        raw = xs * self.sx_scale + self.sx_mean
        S, K, T, r, sg, ic = raw[:, 0], raw[:, 1], torch.clamp(raw[:, 2], min=1e-6), raw[:, 3], raw[:, 4], raw[:, 5]
        f = torch.stack([torch.log(S / K), torch.sqrt(T), sg, r, ic, sg * torch.sqrt(T)], dim=1)
        y = torch.clamp(torch.exp(self.core((f - self.c_mean) / self.c_std)) - EPS, min=0.0)
        return (y / 100.0 * S).unsqueeze(1)

Fs_tr = core_feats(synth_tr)
C_MEAN, C_STD = Fs_tr.mean(0), Fs_tr.std(0) + 1e-8
T_ = lambda a: torch.tensor(a, dtype=torch.float32, device=device)
prep = lambda d: T_((core_feats(d) - C_MEAN) / C_STD)
Xs_tr, ys_tr, Xs_va, ys_va = prep(synth_tr), T_(target(synth_tr)), prep(synth_va), T_(target(synth_va))
Xr_tr, yr_tr, Xr_va, yr_va = prep(real_tr), T_(target(real_tr)), prep(real_va), T_(target(real_va))
print("Tensors ready:", Xs_tr.shape, Xr_tr.shape)
```

4.6.6 Two-stage training (Colab cell 6)

Listing 4.6: Training loop, pre-training and fine-tuning

```
# Cell 6 - Stage 1 (pre-train on synthetic BS) and Stage 2 (fine-tune on real snapshots)
import copy
def train_loop(core, epoch_data, Xv, yv, epochs, lr, tag):
    opt = torch.optim.AdamW(core.parameters(), lr=lr, weight_decay=1e-4)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs, eta_min=lr * 0.01)
    crit = nn.HuberLoss(delta=0.5)
    best, state, bad, hist = float("inf"), None, 0, {"train": [], "val": []}
    for ep in range(1, epochs + 1):
        core.train(); X, y = epoch_data(); perm = torch.randperm(len(X), device=device); tot = 0.0
        for i in range(0, len(X), CFG["batch"]):
            idx = perm[i:i + CFG["batch"]]
            if len(idx) < 2: continue                       # BatchNorm needs > 1 row
            opt.zero_grad(); loss = crit(core(X[idx]), y[idx]); loss.backward(); opt.step()
```

```
            tot += loss.item() * len(idx)
        sched.step(); core.eval()
        with torch.no_grad(): vl = crit(core(Xv), yv).item()
        hist["train"].append(tot / len(X)); hist["val"].append(vl)
        if vl < best: best, state, bad = vl, copy.deepcopy(core.state_dict()), 0
        else: bad += 1
        if ep == 1 or ep % 5 == 0: print(f"[{tag}] epoch {ep:03d} | train {hist['train'][-1]:.5f} | val {vl:.5f} | best {best:.5f}")
        if bad >= CFG["patience"]: print(f"[{tag}] early stop at epoch {ep}"); break
    core.load_state_dict(state); core.eval()
    return hist

core_pre = Core().to(device)
hist_pre = train_loop(core_pre, lambda: (Xs_tr, ys_tr), Xs_va, ys_va, CFG["pre_epochs"], CFG["lr_pre"], "pretrain")

core_ft = copy.deepcopy(core_pre)
def ft_epoch():                                          # real rows + replayed synthetic rows
    k = int(CFG["replay"] * len(Xr_tr)); idx = torch.randint(0, len(Xs_tr), (k,), device=device)
    return torch.cat([Xr_tr, Xs_tr[idx]]), torch.cat([yr_tr, ys_tr[idx]])
hist_ft = train_loop(core_ft, ft_epoch, Xr_va, yr_va, CFG["ft_epochs"], CFG["lr_ft"], "finetune")
```

4.6.7 Export and backend-path check (Colab cell 8, excerpt)

Listing 4.7: TorchScript export and backend consistency check

```
core_dev = core_ft if choice == "NN fine-tuned" else core_pre     # stays on 'device' for the notebook-side check
core_final = copy.deepcopy(core_dev).to("cpu").eval()              # CPU copy used only for export
print("Exporting:", choice)

sx = StandardScaler().fit(pd.concat([real_tr[FEATURE_COLS], synth_tr[FEATURE_COLS]]).values)
sy = StandardScaler(); sy.mean_ = np.zeros(1); sy.scale_ = np.ones(1); sy.var_ = np.ones(1)
sy.n_features_in_ = 1; sy.n_samples_seen_ = 1                  # identity: the model already outputs USD
pricer = Pricer(core_final, C_MEAN, C_STD, sx.mean_, sx.scale_).eval()
ex = torch.tensor(sx.transform(real_te[FEATURE_COLS].values[:8]), dtype=torch.float32)
torch.jit.trace(pricer, ex).save(f"{OUT}/best_model_scripted.pt")
torch.save(core_final.state_dict(), f"{OUT}/best_model.pt")
joblib.dump(sx, f"{OUT}/scaler_X.pkl"); joblib.dump(sy, f"{OUT}/scaler_y.pkl")

# replicate exactly what backend/main.py does and compare
m = torch.jit.load(f"{OUT}/best_model_scripted.pt", map_location="cpu"); m.eval()
sx2, sy2 = joblib.load(f"{OUT}/scaler_X.pkl"), joblib.load(f"{OUT}/scaler_y.pkl")
def backend_price(p):
    mon = p["S"] / p["K"]
    f = np.array([[p["S"], p["K"], p["T"], p["r"], p["sigma"], p["is_call"], mon, float(np.log(mon)), float(np.sqrt(max(p["T"], 1e-8)))]])
    with torch.no_grad(): out = m(torch.tensor(sx2.transform(f), dtype=torch.float32))
    return float(sy2.inverse_transform(out.numpy())[0, 0])
chk = real_te.sample(6, random_state=1)
rows = [dict(inst=r_.instrument, backend_nn=backend_price(r_.to_dict()), notebook_nn=float(predict_usd(core_dev, chk.loc[[i]])[0]),
             bs=float(bs_on(chk.loc[[i]])[0]), market_mid=r_.mid_price) for i, r_ in chk.iterrows()]
chk_df = pd.DataFrame(rows); print(chk_df.round(2))
assert np.allclose(chk_df.backend_nn, chk_df.notebook_nn, rtol=2e-3, atol=1e-2), "Backend path != notebook path"
print("Backend compatibility check passed.")

json.dump(dict(config=CFG, exported=choice, raw_rows=int(len(raw)), clean_rows=int(len(real)),
               snapshots=int(raw.snap_no.nunique()), cleaning=clean_log,
               split_rows={k: int(len(x)) for k, x in dict(train=real_tr, val=real_va, test=real_te).items()},
               real_val=res["val (unseen expiries)"].to_dict(), real_test=res["test (unseen expiries)"].to_dict(),
```

```
                synthetic_fidelity=m_fid), open(f"{OUT}/metrics.json", "w"), indent=2, default=float)
json.dump(dict(pretrain=hist_pre, finetune=hist_ft), open(f"{OUT}/history.json", "w"))
print("Saved to", OUT, ":", sorted(os.listdir(OUT)))
```

4.6.8 Backend inference (`backend/main.py`)

Listing 4.8: Feature construction and price endpoint helper

```
def build_features(p):
    mon = p["S"] / p["K"]
    return np.array([[
        p["S"], p["K"], p["T"], p["r"], p["sigma"], p["is_call"],
        mon, float(np.log(mon)), float(np.sqrt(max(p["T"], 1e-8))),
    ]])


def nn_infer(features, model, scaler_X, scaler_y):
    X_s = scaler_X.transform(features)
    t = torch.tensor(X_s, dtype=torch.float32)
    with torch.no_grad():
        out = model(t)
    return float(scaler_y.inverse_transform(out.numpy())[0, 0])


def price_one(p):
    """Black-Scholes (+ NN when loaded) for one normalised request dict."""
    model, scaler_X, scaler_y = get_model()
    bs_out = bs_price_and_greeks(p["S"], p["K"], p["T"], p["r"], p["sigma"], p["is_call"])
    bs_p = bs_out["price"]

    result = {
        "bs_price": round(bs_p, 4),
        "greeks": {k: round(v, 6) for k, v in bs_out["greeks"].items()},
        "inputs": p,
    }

    if model is not None:
        nn_p = nn_infer(build_features(p), model, scaler_X, scaler_y)
        result["nn_price"] = round(nn_p, 4)
        result["abs_error"] = round(abs(nn_p - bs_p), 6)
        result["pct_error"] = round(abs(nn_p - bs_p) / bs_p * 100, 4) if bs_p > 0 else 0
    else:
        result["nn_price"] = None
        result["abs_error"] = None
        result["pct_error"] = None
        result["note"] = "Model not loaded - showing BS price only. Train the model first."
    return result
```

4.7 Output

4.7.1 Training output on Google Colab

Fig. 4.4 is the output produced by the Colab notebook (cell 8, `eval.png`) after training on the T4 GPU. The left panel shows the Stage 1 training and validation loss on synthetic data over 100 epochs, the centre panel shows Stage 2 fine-tuning on real quotes over 60 epochs with validation on unseen expiries, and the right panel plots the network price against the market mid price for the test expiry on logarithmic axes; the points lie on the red identity line over five orders of magnitude (USD 1 to above USD 20,000).

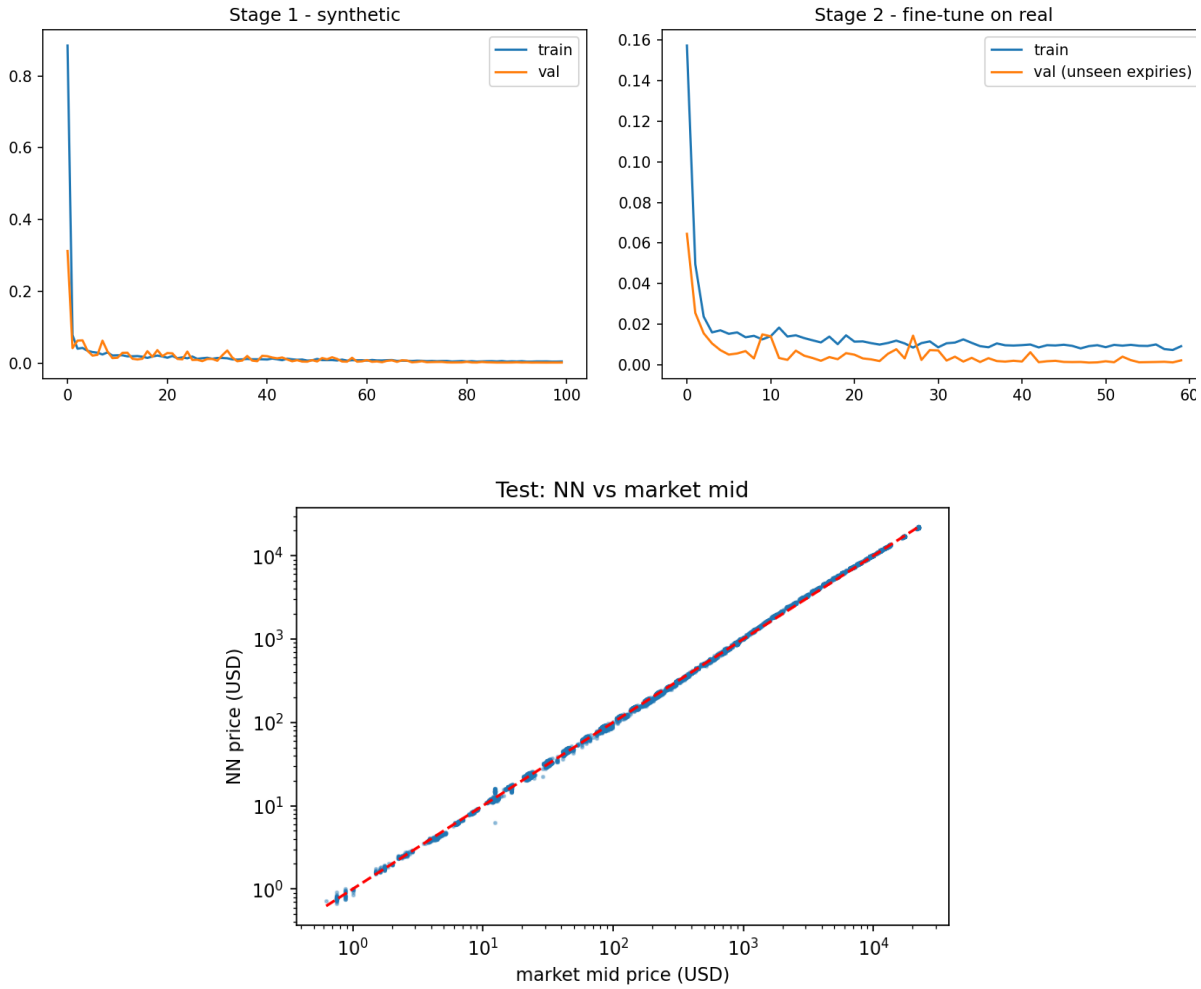


Figure 4.4: Google Colab Training Output (Training Curves and NN vs Market Mid Price)

4.7.2 Model performance on real quotes

Table 4.10 gives the results on validation and test expiries, and Table 4.11 and Fig. 4.5 break the test error down by currency and moneyness (ITM, ATM when $|\ln(S/K)| <$

0.05, and OTM).

Table 4.10: Model Performance on Real Quotes (unseen expiries)

Set	Model	MAE (USD)	RMSE (USD)	MAE (% S)	Med. rel. (%)	R^2
Validation (20,627)	BS (mark IV)	102.18	201.80	0.1376	0.98	0.9998
	NN pre-trained	129.27	252.33	0.2834	1.59	0.9996
	NN fine-tuned	122.29	278.50	0.2755	1.49	0.9995
Test (6,754)	BS (mark IV)	23.15	39.52	0.0415	0.92	0.9999
	NN pre-trained	117.82	219.34	0.2758	5.66	0.9974
	NN fine-tuned	80.38	147.67	0.1561	3.39	0.9988

Table 4.11: Test MAE (% of spot) by Currency and Moneyness

Group	BS (mark IV)	NN pre-trained	NN fine-tuned
BTC	0.0497	0.2490	0.1717
ETH	0.0313	0.3093	0.1366
In-the-money (ITM)	0.0763	0.5794	0.2661
At-the-money (ATM)	0.0392	0.1998	0.1849
Out-of-the-money (OTM)	0.0081	0.0343	0.0187

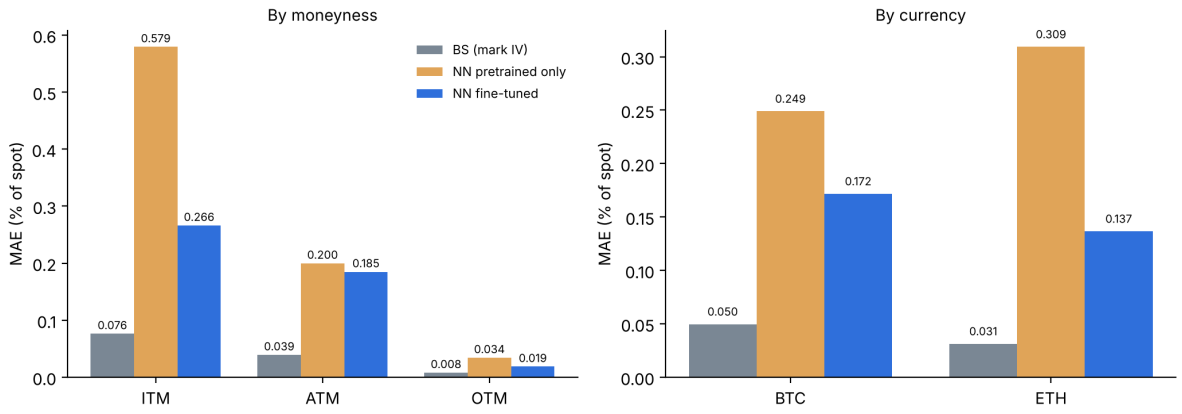


Figure 4.5: Test Error of Black–Scholes and the Neural Networks (MAE as % of spot)

Interpretation. Fine-tuning on real quotes reduces the test MAE of the pre-trained network from USD 117.82 to USD 80.38 and the median relative error from 5.66 % to

3.39 %, and the improvement appears for both currencies and in the ITM and OTM groups. On the validation expiries the fine-tuned network has a slightly lower MAE but a higher RMSE (278.50 against 252.33), which shows that a few large errors remain. Black–Scholes with the exchange's mark volatility is more accurate than both networks, as expected, because the volatility is supplied to it. On fresh synthetic data over the wide range, the fine-tuned network is slightly less faithful to Black–Scholes than the pre-trained one (MAE 1.48 % against 1.16 % of spot, R^2 0.9987 against 0.9995), which is the price paid for adapting to the real market.

4.7.3 Backend-path consistency

After export, the notebook replicates what the backend does (scaler transform → TorchScript model) for six random test contracts and compares it with the notebook prediction. All six agree to within one cent (Table 4.12), and the assertion "Backend compatibility check passed" was printed.

Table 4.12: Backend-Path Consistency Check (prices in USD)

Contract	**Backend NN**	**Notebook NN**	**Black–Scholes**	**Market mid**
ETH-23OCT26-2200-P	12.67	12.67	13.19	13.08
ETH-23OCT26-2650-C	32.17	32.17	30.00	29.27
BTC-23OCT26-95000-C	63.86	63.86	63.97	66.02
ETH-23OCT26-2800-P	308.27	308.27	307.97	309.83
ETH-23OCT26-2850-P	355.77	355.76	358.06	359.34
BTC-23OCT26-105000-P	21,914.47	21,914.46	22,293.40	22,203.33

4.7.4 User interface screens

The following screens of the NeuralPricer web application were captured while the Flask API, MongoDB and the trained model were running.

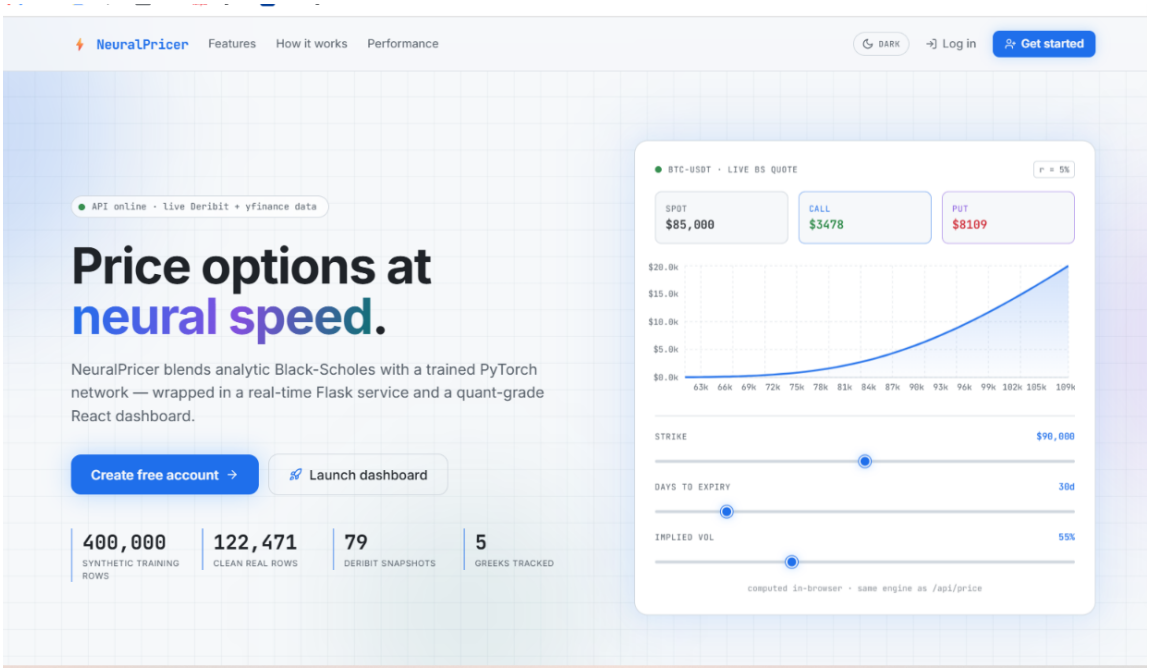


Figure 4.6: Landing Page

The landing page introduces NeuralPricer, shows that the API is online with live Deribit data, and includes an interactive in-browser Black–Scholes quote with strike, expiry and volatility sliders. The counters show 400,000 synthetic training rows, 122,471 clean real rows, 79 Deribit snapshots and 5 Greeks.

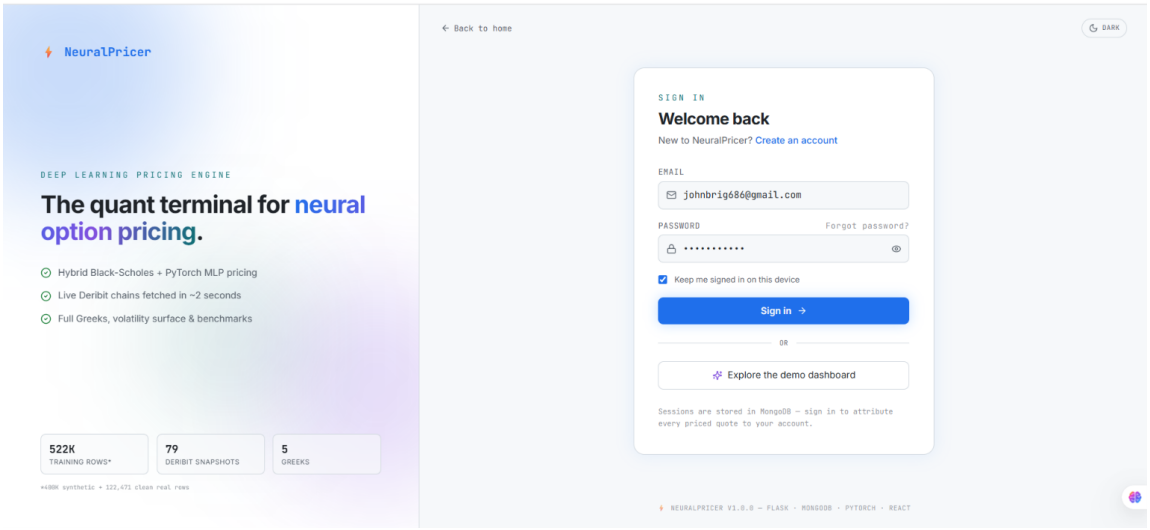


Figure 4.7: Sign-In Page

Users sign in with e-mail and password; sessions are stored in MongoDB so that every priced quote is attributed to the account. A demo dashboard can be explored without signing in.

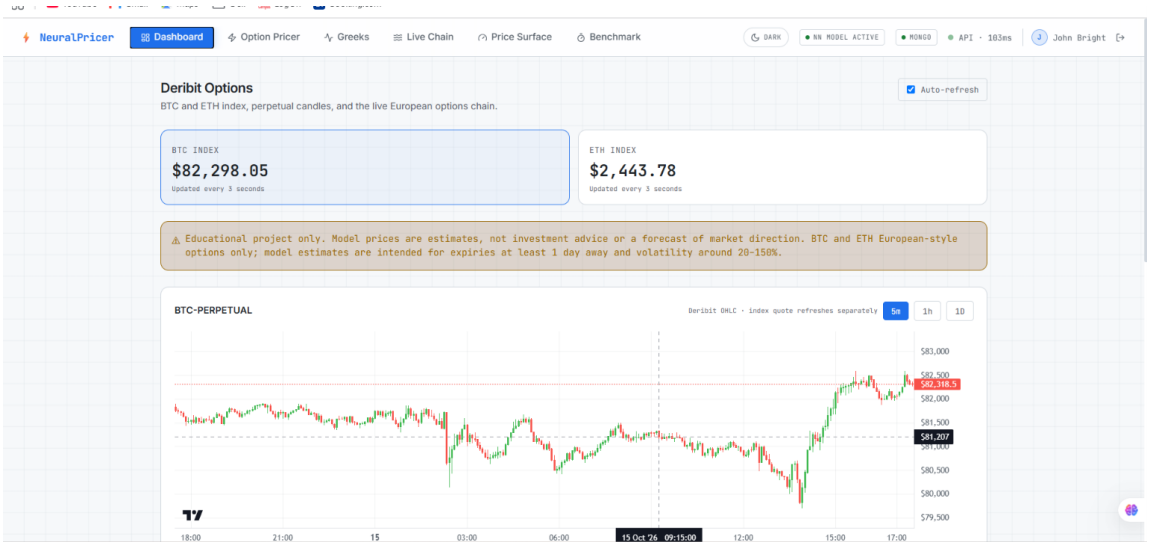


Figure 4.8: Dashboard – Deribit Index and BTC-Perpetual Chart

The dashboard shows the live BTC and ETH index prices, refreshed every few seconds, an educational disclaimer and a BTC-perpetual candlestick chart with 5-minute, 1-hour and 1-day views. The header shows that the NN model and MongoDB are active and the API latency.

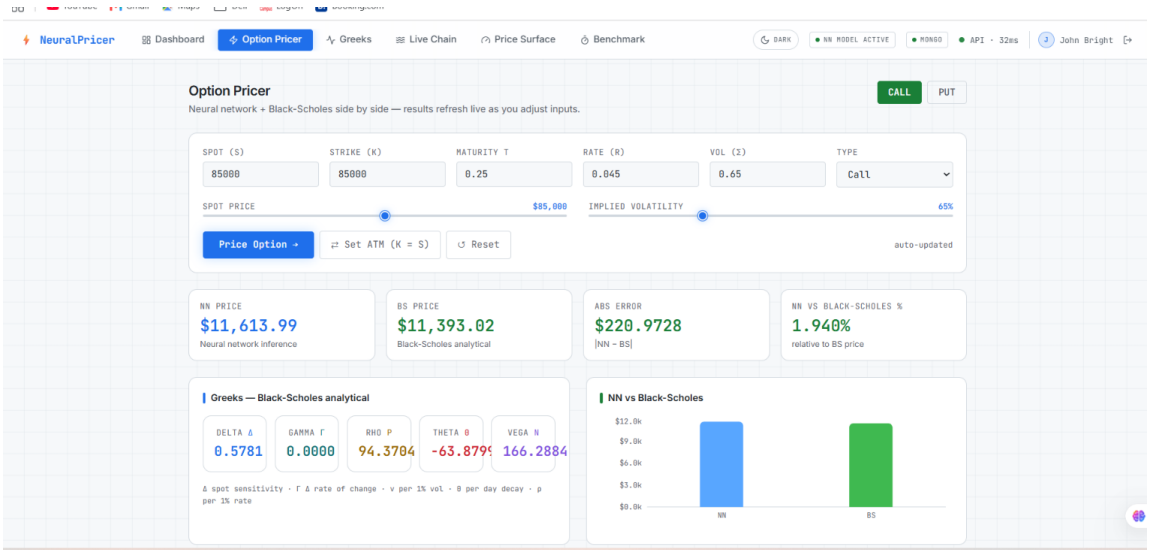


Figure 4.9: Option Pricer (Neural Network vs Black–Scholes)

For $S = K = 85{,}000$, $T = 0.25$, $r = 4.5\,\%$ and $\sigma = 65\,\%$ the neural network returns USD 11,613.99 against the Black–Scholes price of USD 11,393.02, an absolute difference of USD 220.97 (1.94 %). The panel also shows the analytic Greeks and a bar chart of the two prices.

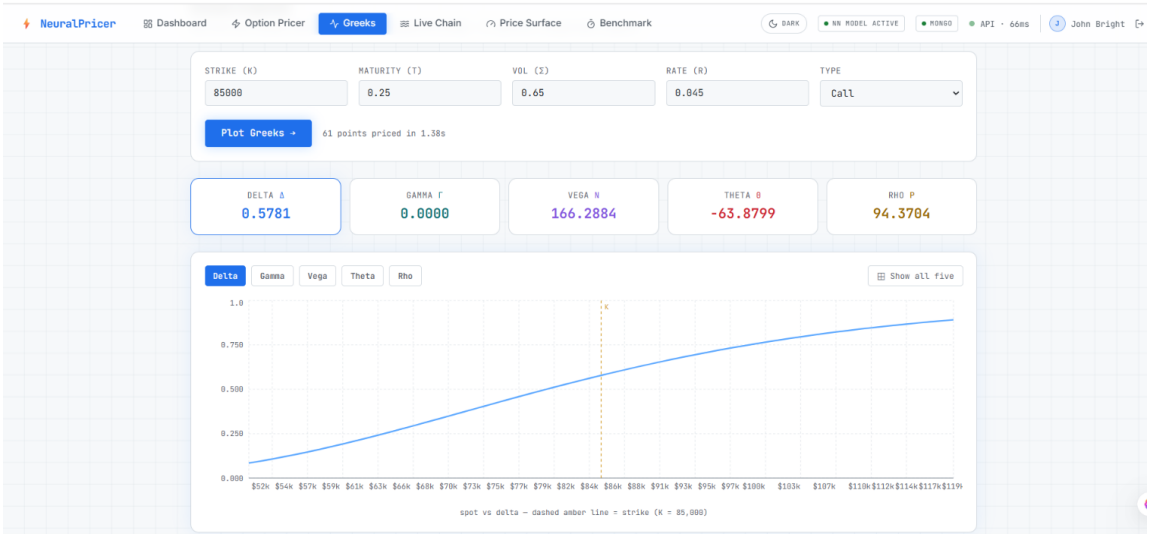


Figure 4.10: Greeks Page

The Greeks page plots Delta, Gamma, Vega, Theta or Rho against the spot price for the chosen strike, maturity, volatility and rate; 61 points were priced in 1.38 seconds. Delta of the 85,000 call is 0.5781 and vega is 166.2884.

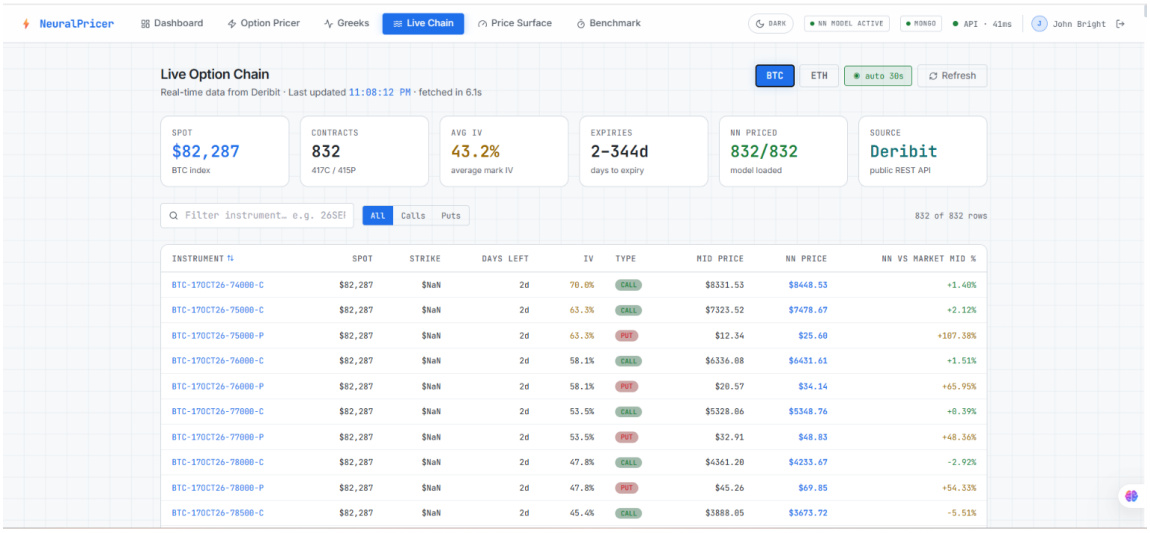


Figure 4.11: Live Option Chain from Deribit

The live chain fetches the BTC (or ETH) chain from Deribit and lists mark implied volatility, exchange mid price, neural-network price and the percentage difference between the two. The header summarises spot, number of contracts (832), average mark IV (43.2 %), expiries and the number of contracts priced by the model (832 of 832).

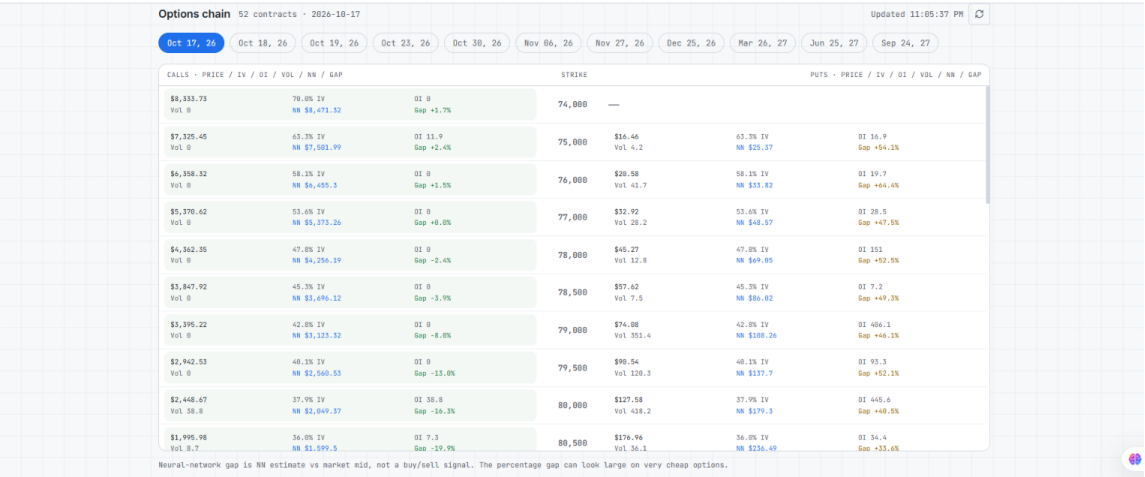


Figure 4.12: Expiry-wise Options Chain with NN–Market Gap

The expiry-wise chain presents calls and puts side by side around each strike with price, implied volatility, open interest and the NN estimate. The note below the table explains that the percentage gap is a model-versus-market signal, not a buy or sell signal, and can look large for very cheap options.

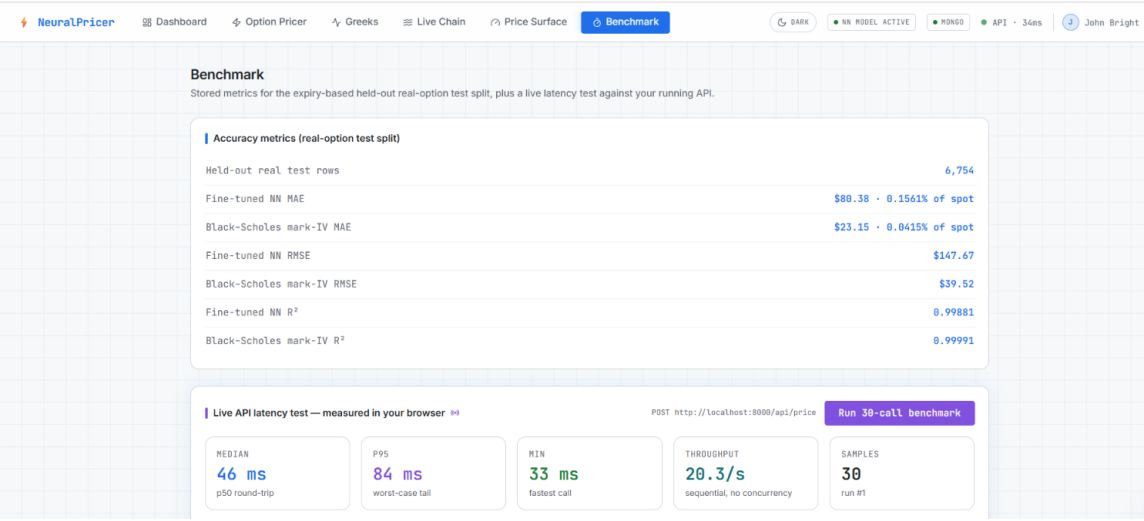


Figure 4.13: Benchmark Page

The benchmark page reproduces the stored test metrics of Table 4.10 (6,754 held-out rows; NN MAE USD 80.38, Black–Scholes MAE USD 23.15; R^2 0.99881 and 0.99991) and runs a live latency test of 30 sequential `POST /api/price` calls measured in the browser: median 46 ms, 95th percentile 84 ms, minimum 33 ms and 20.3 calls per second. These figures include the browser–server round trip and are not the speed of the neural network alone.

4.8 Test Cases and Validation

Table 4.13: Representative Test Cases

ID	Test	Check	Result
T1	Known call value	$S = K = 100, T = 1, r = 5\%, \sigma = 20\%$ gives 10.45058	Pass
T2	Known put value	Same inputs give 5.57353	Pass
T3	Put–call parity	$C - P = S - Ke^{-rT}$ within 10^{-8} ($S = 85{,}000$, $K = 83{,}000$)	Pass
T4	Delta bounds	Call delta in (0,1), put delta in (-1,0), difference 1	Pass
T5	Non-negativity and Greeks	Deep out-of-the-money call price $\geq$ 0; all five Greeks returned	Pass
T6	Degenerate inputs	$T = 0$ and $\sigma = 0$ do not crash	Pass
T7	Backend-path consistency	Six random test contracts: backend NN equals notebook NN (rtol 2×10^{-3})	Pass
T8	Hold-out integrity	Validation and test expiries are disjoint from training expiries; no split empty	By design
T9	API latency	30 sequential price calls: median 46 ms, p95 84 ms	Measured

Tests T1–T6 are the six unit tests in `tests/test_black_scholes.py`; all six passed when run for this report.

Validation and limitations.

- The real data cover about one percent of spot movement and a short period, so the real-market evidence comes from one market regime; the test set is a single expiry per currency (6,754 quotes), and results may differ for other days.
- The neural pricer is less accurate than Black–Scholes with mark implied volatility (USD 80.38 against USD 23.15), because the exchange's volatility is an input to

the analytic formula. The value of the network lies in speed, a single shared model and differentiability, not in beating the formula.

- Errors are larger for in-the-money options and a few large errors remain (validation RMSE of the fine-tuned network is higher than that of the pre-trained one).
- The risk-free rate is fixed at 4.5 %, options are treated as European and only BTC and ETH are covered.
- Greeks in the application are the analytic Black–Scholes Greeks; automatic differentiation of the network has not been added yet.
- The latency figures were measured through the browser and include network overhead; model-only latency has not been benchmarked separately.

4.9 Conclusion and Future Scope

NeuralPricer demonstrates a complete pipeline for learning an option pricing surface from real exchange data: live collection of 79 Deribit snapshots, leakage-aware cleaning and expiry-based splitting, wide-range synthetic pre-training, scale-free two-stage learning, TorchScript export, a Flask and MongoDB backend and a React dashboard. On 6,754 quotes from unseen expiries the fine-tuned network reached MAE USD 80.38 (0.156 % of spot) and $R^2 = 0.9988$, a 31.8 % lower MAE than the synthetic-only network, while a single 110,209-parameter model priced both BTC and ETH.

Future work includes: (i) collecting snapshots over a longer period and over larger spot moves; (ii) computing Greeks by automatic differentiation of the network and adding a Black–Scholes PDE residual as a physics-informed loss; (iii) learning the implied-volatility surface instead of taking implied volatility as an input; (iv) training on richer synthetic data from stochastic-volatility or jump models; (v) extending to equity options through yfinance and to American-style exercise; and (vi) reporting model-only inference latency and calibrated error bands.

REFERENCES

1. F. Black and M. Scholes, "The Pricing of Options and Corporate Liabilities," *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973.

2. R. C. Merton, "Theory of Rational Option Pricing," *Bell Journal of Economics and Management Science*, vol. 4, no. 1, pp. 141–183, 1973.

3. J. M. Hutchinson, A. W. Lo, and T. Poggio, "A Nonparametric Approach to Pricing and Hedging Derivative Securities via Learning Networks," *The Journal of Finance*, vol. 49, no. 3, pp. 851–889, 1994.

4. R. Culkin and S. R. Das, "Machine Learning in Finance: The Case of Deep Learning for Option Pricing," *Journal of Investment Management*, vol. 15, no. 4, pp. 92–100, 2017.

5. S. Liu, C. W. Oosterlee, and S. M. Bohte, "Pricing Options and Computing Implied Volatilities Using Neural Networks," *Risks*, vol. 7, no. 1, article 16, 2019.

6. B. Horvath, A. Muguruza, and M. Tomas, "Deep Learning Volatility: A Deep Neural Network Perspective on Pricing and Calibration in (Rough) Volatility Models," *Quantitative Finance*, vol. 21, no. 1, pp. 11–27, 2021.

7. J. Ruf and W. Wang, "Neural Networks for Option Pricing and Hedging: A Literature Review," *Journal of Computational Finance*, vol. 24, no. 1, 2020.

8. M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

9. S. L. Heston, "A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options," *The Review of Financial Studies*, vol. 6, no. 2, pp. 327–343, 1993.

10. J. C. Hull, *Options, Futures, and Other Derivatives*, 11th ed., Pearson, 2021.

11. D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)," *arXiv:1511.07289*, 2015.

12. S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. ICML*, 2015.

13. I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *Proc. ICLR*, 2019.

14. P. J. Huber, "Robust Estimation of a Location Parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

15. A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

16. Deribit, "Deribit API v2 Documentation – Public Market Data Methods," https://docs.deribit.com.

17. NeuralPricer project repository (source code, Colab notebook, trained model and metrics), https://github.com/dinesh90040/NeuralPricer.

APPENDIX

Appendix A: Key Dataset Numbers

- Snapshots = 79; raw rows = 127,322 (BTC 68,279; ETH 59,043); unique contracts = 1,748.
- Clean real rows = 122,471 (train 95,090; validation 20,627; test 6,754).
- Synthetic samples = 400,000 generated, 278,719 kept (train 250,848; validation 27,871).
- BTC spot range = 82,334 – 83,388; ETH spot range = 2,488 – 2,512.

Appendix B: Final Model Configuration

- Network = MLP 6–256–256–128–64–1 (BatchNorm, ELU), 110,209 parameters.
- Target = $\ln(100V/S + 0.01)$; Huber loss ($\delta = 0.5$); AdamW (weight decay 10^{-4}); cosine annealing; batch 4,096.
- Stage 1 = 100 epochs at 2×10^{-3}; Stage 2 = 60 epochs at 2×10^{-4} with replay ratio 1.0; seed 42.
- Exported model = fine-tuned network (selected on validation MAE).

Appendix C: Final Test Performance

- NN fine-tuned: MAE USD 80.38 (0.1561 % of spot), RMSE USD 147.67, $R^2 = 0.9988$, median relative error 3.39 %.
- Black–Scholes (mark IV): MAE USD 23.15, RMSE USD 39.52, $R^2 = 0.9999$.
- Improvement of fine-tuned over pre-trained network: 31.8 % lower MAE and 32.7 % lower RMSE on the test expiry.
- Median API round-trip latency measured in the browser: 46 ms (30 calls).

Appendix D: Project Structure

```
NeuralPricer/
  backend/    main.py (Flask API), black_scholes.py, data_collector.py
  training/   generate_dataset.py, train.py
  frontend/   React + Vite dashboard (pages: Landing, Login, Signup,
              Pricer, Greeks, LiveChain, Surface, Benchmark)
  models/     best_model_scripted.pt, best_model.pt, scaler_X.pkl,
              scaler_y.pkl, metrics.json, history.json, eval.png
  tests/      test_black_scholes.py
```

Appendix E: How to Run

```
# 1. Backend (Python 3.10+, MongoDB running)
cd backend
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python main.py                 # API on http://127.0.0.1:8000

# 2. Frontend
cd frontend
npm install
npm run dev                    # dashboard on http://localhost:3000

# 3. Tests
python tests/test_black_scholes.py
```